

Методический материал для студентов

Decision Tree и Random Forest после метрик качества моделей

Практический урок по машинному обучению на Python и scikit-learn

Назначение: Материал рассчитан на студентов, которые уже прошли Linear Regression, Logistic Regression, KNN и базовые метрики качества классификации. Цель занятия - связать оценку качества моделей с новыми алгоритмами: Decision Tree и Random Forest.

1. Цели урока

После занятия студент должен уметь:

1. объяснять, зачем нужны метрики качества модели;
2. читать confusion matrix и различать TP, TN, FP, FN;
3. рассчитывать и интерпретировать accuracy, precision, recall и F1-score;
4. объяснять принцип работы Decision Tree;
5. отличать Decision Tree от Logistic Regression и KNN;
6. понимать, почему дерево решений может переобучаться;
7. использовать max_depth и другие ограничения дерева;
8. объяснять принцип работы Random Forest;
9. обучать DecisionTreeClassifier и RandomForestClassifier в scikit-learn;
10. сравнивать Logistic Regression, KNN, Decision Tree и Random Forest по метрикам.

2. Что мы уже знаем перед этим уроком

Модель	Для чего используется	Главная идея
Linear Regression	Предсказание числа	Строит линейную зависимость между признаками и числовым ответом.
Logistic Regression	Классификация	Считает вероятность принадлежности к классу.
KNN	Классификация	Смотрит на ближайших соседей объекта.
Метрики качества	Оценка модели	Показывают, где модель права, а где ошибается.

Теперь мы добавляем модели, которые работают через правила и коллективное голосование: Decision Tree и Random Forest.

3. Сквозная задача урока

На протяжении урока будем решать одну задачу: предсказать, сдаст студент курс или нет.

Колонка	Смысл
attendance	Посещаемость в процентах.
homeworks_done	Количество выполненных домашних заданий.
test_score	Средний балл за тест.
practice_hours	Количество часов практики.
missed_classes	Количество пропущенных занятий.
activity_score	Активность на занятии.

passed	Целевая переменная: 1 - сдал, 0 - не сдал.
--------	--

Ключевой термин: Целевая переменная - это то, что модель должна предсказывать. В нашем случае это passed.

4. Повторение метрик качества классификации

4.1. Зачем нужны метрики

Обучить модель недостаточно. Модель может выдавать предсказания, но нам нужно понять, насколько этим предсказаниям можно доверять. Метрики качества позволяют измерять работу модели численно, а не на уровне ощущения.

Например, модель говорит: студент сдаст курс. Но в реальности студент может сдать или не сдать. Поэтому каждое предсказание модели можно отнести к правильному или ошибочному.

4.2. Confusion Matrix

Confusion Matrix - это таблица ошибок модели. Для бинарной классификации есть четыре варианта результата.

Термин	Расшифровка	Пример в задаче студентов
TP	True Positive	Модель сказала "сдаст", и студент реально сдал.
TN	True Negative	Модель сказала "не сдаст", и студент реально не сдал.
FP	False Positive	Модель сказала "сдаст", но студент не сдал.
FN	False Negative	Модель сказала "не сдаст", но студент сдал.

Важно: В sklearn confusion_matrix для бинарной классификации обычно выводится в порядке [[TN, FP], [FN, TP]].

4.3. Accuracy

Accuracy - это доля правильных ответов модели среди всех ответов.

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Пример: если из 100 студентов модель правильно предсказала результат для 85, то accuracy = 85 / 100 = 0.85, то есть 85%.

Accuracy хорошо работает, когда классы сбалансированы: например, 50 студентов сдали и 50 не сдали.

Но accuracy может обманывать. Если 95 студентов сдали, а 5 не сдали, модель может всегда говорить "сдаст" и получить accuracy 95%, хотя она вообще не умеет находить тех, кто не сдаст.

4.4. Precision

Precision показывает, насколько можно доверять положительным предсказаниям модели.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Если модель сказала, что 40 студентов сдадут, а реально из них сдали 30, то precision = 30 / 40 = 0.75. Это значит, что когда модель говорит "сдаст", она права в 75% случаев.

Precision особенно важен, когда ложное положительное предсказание дорого обходится. Например, модель сказала, что студент готов, но на самом деле он не готов, и преподаватель перестал обращать на него внимание.

4.5. Recall

Recall показывает, сколько реальных объектов положительного класса модель смогла найти.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Если реально сдали 50 студентов, а модель нашла 40 из них, то $\text{recall} = 40 / 50 = 0.8$. Это значит, что модель нашла 80% всех студентов, которые реально сдали.

Recall особенно важен, когда нельзя пропустить нужный класс. Например, если положительный класс - студент в зоне риска, низкий recall означает, что модель пропускает много студентов, которым нужна помощь.

4.6. F1-score

F1-score - это баланс между precision и recall. Он полезен, когда нельзя оценивать модель только по одной из этих метрик.

$$\text{F1} = 2 * (\text{precision} * \text{recall}) / (\text{precision} + \text{recall})$$

Если F1-score высокий, значит модель не только достаточно точная, но и хорошо находит нужный класс.

4.7. Classification Report

В sklearn можно вывести основные метрики одной командой:

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

Classification report показывает precision, recall, f1-score и support. Support - это количество объектов каждого класса в тестовой выборке.

4.8. Мини-практика по метрикам

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

y_test = [1, 1, 1, 0, 0, 0, 1, 0, 1, 0]
y_pred = [1, 1, 0, 0, 1, 0, 1, 0, 1, 0]

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1-score:", f1_score(y_test, y_pred))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

Разбор: `y_test` - реальные ответы, `y_pred` - предсказания модели. Функции из `sklearn` сравнивают эти два списка и рассчитывают метрики.

5. Decision Tree: дерево решений

5.1. Что такое Decision Tree

Decision Tree - это модель машинного обучения, которая принимает решение через последовательность условий. По смыслу она очень похожа на цепочку `if/else`.

```
if test_score >= 60:
    if attendance >= 70:
        prediction = 1
    else:
        prediction = 0
else:
    prediction = 0
```

Decision Tree делает похожую вещь, но условия выбирает автоматически на основе данных.

Пример логики дерева:

```
test_score >= 60?
|-- yes:
|   |-- attendance >= 70?
|   |   |-- yes: passed
|   |   |-- no: not passed
|-- no:
|   |-- not passed
```

5.2. Основные элементы дерева

Элемент	Объяснение	Пример
Root node	Корневой узел, первый вопрос дерева.	<code>test_score <= 57.5?</code>
Internal node	Промежуточный узел, следующий вопрос.	<code>attendance <= 68.5?</code>
Branch	Ветка после ответа на вопрос.	<code>True / False</code>
Leaf node	Лист, финальное решение.	<code>class = passed</code>
Depth	Глубина дерева, количество уровней вопросов.	<code>max_depth = 3</code>
Split	Разделение данных по условию.	<code>test_score <= 60</code>

5.3. Как дерево выбирает вопросы

Дерево ищет такой признак и такой порог, которые лучше всего разделяют данные на классы. Оно пытается сделать так, чтобы после разделения в каждой группе было как можно больше объектов одного класса.

Например, если в группе 10 студентов: 5 сдали и 5 не сдали, группа грязная, потому что классы перемешаны. Если после разделения получилась группа, где 9 сдали и 1 не сдал, такая группа намного чище.

Состав узла	Оценка
5 сдали, 5 не сдали	Плохой узел: классы перемешаны.
9 сдали, 1 не сдал	Хороший узел: почти все одного класса.
10 сдали, 0 не сдали	Идеальный узел: полностью чистый.

5.4. Gini impurity

Gini impurity показывает, насколько классы перемешаны в узле. Если $Gini = 0$, узел полностью чистый. Чем выше Gini, тем сильнее классы перемешаны.

$$Gini = 1 - p_0^2 - p_1^2$$

Где p_0 - доля класса 0, p_1 - доля класса 1.

Пример: в узле 10 студентов, 8 сдали и 2 не сдали. Тогда $p_1 = 0.8$, $p_0 = 0.2$.

$$Gini = 1 - 0.8^2 - 0.2^2$$

$$Gini = 1 - 0.64 - 0.04$$

$$Gini = 0.32$$

Если в узле 10 сдали и 0 не сдали, тогда $p_1 = 1$, $p_0 = 0$, $Gini = 0$. Это полностью чистый узел.

5.5. Отличие Decision Tree от Logistic Regression

Критерий	Logistic Regression	Decision Tree
Логика	Математическая формула и вероятность.	Цепочка условий.
Граница решения	Чаще линейная.	Ступенчатая, может быть нелинейной.
Масштабирование	Желательно.	Обычно не нужно.
Интерпретация	Через веса и вероятность.	Через правила дерева.
Риск переобучения	Умеренный.	Высокий без ограничений.

Logistic Regression пытается построить границу через взвешенную сумму признаков. Decision Tree не строит одну прямую, а делит данные по условиям: $test_score \leq 60$, $attendance \leq 70$ и так далее.

5.6. Отличие Decision Tree от KNN

Критерий	KNN	Decision Tree
Как работает	Ищет ближайших соседей.	Строит правила и условия.
Нужен StandardScaler	Да, очень желательно.	Обычно нет.
Скорость на больших данных	Может быть медленнее.	Предсказывает быстро.
Интерпретация	Слабая.	Хорошая.
Главный параметр	$n_neighbors$.	max_depth .

5.7. Почему дереву обычно не нужен StandardScaler

KNN считает расстояния, поэтому признаки с большим масштабом могут доминировать. Logistic Regression тоже часто работает стабильнее после масштабирования. Decision Tree не считает расстояния. Оно задает вопросы вида $test_score \leq 60$ или $attendance \leq 70$. Поэтому масштаб признаков обычно не критичен.

5.8. Overfitting у Decision Tree

Decision Tree может переобучаться. Это значит, что дерево слишком хорошо запоминает тренировочные данные и хуже работает на новых данных.

Если глубину дерева не ограничить, в sklearn будет $max_depth=None$. Тогда дерево будет расти, пока может делить данные. На train качество может стать очень высоким, иногда почти 1.0, но на test качество может ухудшиться.

Важно: Без ограничения `max_depth` дерево может не искать общие закономерности, а запоминать конкретные примеры из `train`.

Пример плохого правила слишком глубокого дерева:

```
if attendance == 73 and test_score == 61 and homeworks_done == 5:  
    prediction = 1
```

Такое правило похоже на запоминание конкретного студента, а не на реальную закономерность.

5.9. Как бороться с переобучением дерева

Параметр	Что делает
<code>max_depth</code>	Ограничивает глубину дерева.
<code>min_samples_split</code>	Минимальное количество объектов для разделения узла.
<code>min_samples_leaf</code>	Минимальное количество объектов в листе.
<code>max_leaf_nodes</code>	Максимальное количество листьев.
<code>criterion</code>	Критерий качества разделения: <code>gini</code> или <code>entropy</code> .

Для первого знакомства чаще всего достаточно показать `max_depth`. Например, `max_depth=3` означает, что дерево может иметь максимум 3 уровня вопросов.

6. Практика: DecisionTreeClassifier

6.1. Импорты

```
import pandas as pd  
import matplotlib.pyplot as plt  
  
from sklearn.model_selection import train_test_split  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.tree import plot_tree  
  
from sklearn.metrics import accuracy_score  
from sklearn.metrics import precision_score  
from sklearn.metrics import recall_score  
from sklearn.metrics import f1_score  
from sklearn.metrics import confusion_matrix  
from sklearn.metrics import classification_report
```

6.2. Создание датасета

```
data = {  
    "attendance": [90, 85, 40, 60, 75, 30, 95, 50, 80, 45,  
                  88, 35, 70, 65, 92, 55, 78, 20, 83, 58],  
    "homeworks_done": [9, 8, 2, 4, 7, 1, 10, 3, 8, 2,  
                      9, 1, 6, 5, 10, 4, 7, 0, 8, 3],  
    "test_score": [88, 80, 35, 55, 72, 25, 95, 45, 78, 40,  
                  85, 30, 68, 60, 90, 50, 74, 20, 82, 48],  
    "practice_hours": [20, 18, 5, 8, 15, 3, 25, 6, 17, 4,  
                      19, 2, 12, 10, 22, 7, 14, 1, 18, 6],  
    "missed_classes": [1, 2, 10, 6, 3, 12, 0, 8, 2, 9,
```

```

        1, 11, 4, 5, 0, 7, 3, 15, 2, 8],
"activity_score": [9, 8, 3, 5, 7, 2, 10, 4, 8, 3,
        9, 2, 6, 6, 10, 5, 7, 1, 8, 4],
"passed": [1, 1, 0, 0, 1, 0, 1, 0, 1, 0,
        1, 0, 1, 1, 1, 0, 1, 0, 1, 0]
}

```

```

df = pd.DataFrame(data)
print(df)

```

`data` - это словарь, где ключи являются названиями колонок, а значения - списками данных. `pd.DataFrame(data)` превращает словарь в таблицу `pandas`.

6.3. Анализ данных

```

print(df.head())
print(df.info())
print(df.describe())
print(df["passed"].value_counts())

```

`head` показывает первые строки, `info` показывает типы данных и пропуски, `describe` показывает числовую статистику, `value_counts` показывает распределение классов.

6.4. Деление на X и y

```

X = df.drop("passed", axis=1)
y = df["passed"]

```

`X` - признаки, по которым модель будет принимать решение. `y` - правильный ответ, который модель должна научиться предсказывать.

6.5. Train/test split

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

```

`test_size=0.3` означает, что 30% данных уйдет на тест. `random_state=42` фиксирует случайность. `stratify=y` сохраняет пропорцию классов в `train` и `test`.

6.6. Обучение Decision Tree

```

tree_model = DecisionTreeClassifier(
    max_depth=3,
    random_state=42
)

```

```

tree_model.fit(X_train, y_train)

```

DecisionTreeClassifier создает модель дерева решений. max_depth=3 ограничивает глубину дерева, чтобы снизить риск переобучения. fit обучает модель на тренировочных данных.

6.7. Предсказание и метрики

```
y_pred = tree_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1-score:", f1_score(y_test, y_pred))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("Classification Report:")
print(classification_report(y_test, y_pred))
```

predict делает предсказания на тестовых данных. Затем мы считаем метрики и анализируем, где модель ошиблась.

6.8. Визуализация дерева

```
plt.figure(figsize=(18, 10))

plot_tree(
    tree_model,
    feature_names=X.columns,
    class_names=["not passed", "passed"],
    filled=True,
    rounded=True
)

plt.show()
```

plot_tree рисует дерево. feature_names показывает названия признаков, class_names показывает названия классов, filled=True раскрашивает узлы, rounded=True делает блоки аккуратнее.

7. Random Forest

7.1. Что такое Random Forest

Random Forest - это ансамблевая модель, которая состоит из множества деревьев решений. Одно дерево может ошибаться и быть нестабильным. Random Forest строит много деревьев и объединяет их ответы голосованием.

Если 100 деревьев голосуют, итоговое решение обычно стабильнее, чем решение одного дерева.

```
Tree 1: passed
Tree 2: passed
Tree 3: not passed
Tree 4: passed
```

Tree 5: not passed

Final prediction: passed

7.2. Почему Random Forest называется случайным лесом

Forest - потому что это много деревьев. Random - потому что каждое дерево строится немного по-разному: на случайных подвыборках строк и с элементами случайности при выборе признаков.

7.3. Bootstrap sampling

Каждое дерево обучается на случайной выборке строк из train-данных. Некоторые строки могут повторяться, некоторые могут не попасть в конкретную выборку.

Original rows: 1, 2, 3, 4, 5, 6, 7, 8

Tree 1 rows: 1, 2, 2, 4, 5, 7, 7, 8

Tree 2 rows: 1, 3, 3, 4, 6, 6, 8, 8

Tree 3 rows: 2, 2, 5, 5, 6, 7, 8, 8

7.4. Random feature selection

При построении дерева Random Forest не всегда рассматривает все признаки сразу. Разные деревья могут больше опираться на разные признаки. Это делает деревья непохожими друг на друга, а итоговое голосование более полезным.

7.5. Отличие Random Forest от Decision Tree

Критерий	Decision Tree	Random Forest
Количество деревьев	Одно дерево.	Много деревьев.
Стабильность	Ниже.	Выше.
Риск переобучения	Выше.	Обычно ниже.
Интерпретация	Очень хорошая.	Хуже, чем у одного дерева.
Скорость	Быстрее.	Медленнее.
Качество	Может быть средним.	Часто выше.

7.6. Основные параметры Random Forest

Параметр	Значение
n_estimators	Количество деревьев в лесу.
max_depth	Максимальная глубина каждого дерева.
random_state	Фиксирует случайность.
max_features	Сколько признаков рассматривать при разделении.
min_samples_split	Минимальное число объектов для разделения узла.
min_samples_leaf	Минимальное число объектов в листе.

Для первого урока достаточно хорошо разобрать n_estimators и max_depth.

8. Практика: RandomForestClassifier

```
from sklearn.ensemble import RandomForestClassifier
```

```
forest_model = RandomForestClassifier(  
    n_estimators=100,  
    max_depth=5,  
    random_state=42  
)
```

```

forest_model.fit(X_train, y_train)

y_pred_forest = forest_model.predict(X_test)

print("Random Forest Results")
print("Accuracy:", accuracy_score(y_test, y_pred_forest))
print("Precision:", precision_score(y_test, y_pred_forest))
print("Recall:", recall_score(y_test, y_pred_forest))
print("F1-score:", f1_score(y_test, y_pred_forest))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_forest))

print("Classification Report:")
print(classification_report(y_test, y_pred_forest))

```

`n_estimators=100` означает, что модель построит 100 деревьев. `max_depth=5` ограничивает глубину каждого дерева. `fit` обучает модель, `predict` делает предсказания.

9. Сравнение всех изученных моделей

Теперь важно связать новый материал с предыдущими моделями. Сравним Logistic Regression, KNN, Decision Tree и Random Forest на одном датасете.

```

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

log_model = LogisticRegression(random_state=42)
knn_model = KNeighborsClassifier(n_neighbors=5)
tree_model = DecisionTreeClassifier(max_depth=3, random_state=42)
forest_model = RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    random_state=42
)

models = [
    ("Logistic Regression", log_model, X_train_scaled, X_test_scaled),
    ("KNN", knn_model, X_train_scaled, X_test_scaled),
    ("Decision Tree", tree_model, X_train, X_test),
    ("Random Forest", forest_model, X_train, X_test)
]

results = []

```

```

for name, model, X_tr, X_te in models:
    model.fit(X_tr, y_train)
    y_pred = model.predict(X_te)

    results.append({
        "model": name,
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred),
        "recall": recall_score(y_test, y_pred),
        "f1_score": f1_score(y_test, y_pred)
    })

results_df = pd.DataFrame(results)
print(results_df)

```

Для Logistic Regression и KNN используются масштабированные данные, потому что эти модели чувствительны к масштабу признаков. Для Decision Tree и Random Forest используются обычные данные, потому что деревья работают через пороги и обычно не требуют StandardScaler.

Главный вывод: Нельзя выбирать модель только по названию. Нужно обучить несколько моделей, посчитать метрики и выбрать модель под конкретную задачу.

10. Feature importance в Random Forest

Random Forest может показать, какие признаки оказались наиболее важными для модели. Это удобно для анализа данных и объяснения результата.

```

importances = forest_model.feature_importances_

feature_importance_df = pd.DataFrame({
    "feature": X.columns,
    "importance": importances
})

feature_importance_df = feature_importance_df.sort_values(
    by="importance",
    ascending=False
)

print(feature_importance_df)

```

`feature_importances_` возвращает важность каждого признака. Чем выше значение, тем сильнее этот признак участвовал в разделениях деревьев.

```

plt.figure(figsize=(10, 6))

plt.bar(
    feature_importance_df["feature"],
    feature_importance_df["importance"]
)

```

```
plt.xticks(rotation=45)
plt.xlabel("Feature")
plt.ylabel("Importance")
plt.title("Feature Importance in Random Forest")
plt.show()
```

Важно: высокая важность признака не означает абсолютную истину. Это важность внутри данной модели и данного датасета.

11. Предсказание для нового студента

```
new_student = pd.DataFrame({
    "attendance": [72],
    "homeworks_done": [6],
    "test_score": [67],
    "practice_hours": [11],
    "missed_classes": [4],
    "activity_score": [6]
})

prediction = forest_model.predict(new_student)
print(prediction)

proba = forest_model.predict_proba(new_student)
print(proba)
```

predict возвращает итоговый класс. predict_proba возвращает вероятности по классам. Например, `[[0.25, 0.75]]` означает: 25% - не сдаст, 75% - сдаст.

12. Итог урока

На этом уроке мы повторили метрики качества: accuracy, precision, recall, F1-score и confusion matrix. Затем изучили Decision Tree - модель, которая принимает решения через последовательность условий. После этого изучили Random Forest - ансамбль из множества деревьев решений.

Главная профессиональная логика: в машинном обучении нет одной лучшей модели для всех задач. Нужно понять задачу, подготовить данные, обучить несколько моделей, сравнить метрики и выбрать подходящую модель.

Linear Regression -> предсказывает число
Logistic Regression -> классифицирует через вероятность
KNN -> классифицирует через ближайших соседей
Decision Tree -> классифицирует через правила
Random Forest -> объединяет много деревьев и голосует

13. Вопросы для самопроверки

11. Что такое confusion matrix?
12. Чем TP отличается от FP?
13. Чем FN отличается от TN?
14. Почему accuracy может обманывать?

15. Что показывает precision?
16. Что показывает recall?
17. Зачем нужен F1-score?
18. Что такое Decision Tree?
19. Почему Decision Tree похож на if/else?
20. Что такое root node?
21. Что такое leaf node?
22. Что делает max_depth?
23. Что будет, если max_depth не указать?
24. Почему дерево может переобучиться?
25. Чем Decision Tree отличается от Logistic Regression?
26. Чем Decision Tree отличается от KNN?
27. Что такое Random Forest?
28. Почему Random Forest стабильнее одного дерева?
29. Что означает n_estimators?
30. Почему деревьям обычно не нужен StandardScaler?
31. Что такое feature importance?

14. Домашнее задание

Задание 1. Decision Tree с разной глубиной

Измените глубину дерева от 1 до 6 и сравните метрики. Нужно вывести accuracy, precision, recall и F1-score для каждого значения max_depth.

```
for depth in range(1, 7):
    model = DecisionTreeClassifier(
        max_depth=depth,
        random_state=42
    )

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    print("max_depth:", depth)
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Precision:", precision_score(y_test, y_pred))
    print("Recall:", recall_score(y_test, y_pred))
    print("F1:", f1_score(y_test, y_pred))
    print("-" * 40)
```

Письменно ответьте: при какой глубине модель показала лучший результат? Стало ли дерево лучше при увеличении глубины? Есть ли признаки переобучения?

Задание 2. Random Forest с разным количеством деревьев

Обучите Random Forest с количеством деревьев 10, 50, 100 и 200. Сравните метрики.

```
for n in [10, 50, 100, 200]:
    model = RandomForestClassifier(
        n_estimators=n,
```

```

max_depth=5,
random_state=42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("n_estimators:", n)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1:", f1_score(y_test, y_pred))
print("-" * 40)

```

Письменно ответьте: стало ли качество лучше при увеличении количества деревьев? Есть ли смысл всегда ставить очень большое количество деревьев?

Задание 3. Сравнение моделей

Сравните четыре модели: Logistic Regression, KNN, Decision Tree и Random Forest. Создайте таблицу результатов и выберите лучшую модель по F1-score.

В отчете обязательно написать: какая модель лучшая по accuracy, какая по recall, какая по F1-score, и какую модель вы бы выбрали для задачи предсказания успешности студента.

Задание 4. Ответы на теорию

32. Чем Decision Tree отличается от Logistic Regression?
33. Чем Decision Tree отличается от KNN?
34. Почему Decision Tree может переобучиться?
35. Что произойдет, если не указать max_depth?
36. Что делает параметр n_estimators?
37. Почему Random Forest называется ансамблевой моделью?
38. Почему Random Forest обычно стабильнее одного дерева?
39. Почему нельзя смотреть только на accuracy?
40. Когда важнее precision?
41. Когда важнее recall?

Критерии оценивания домашнего задания

Критерий	Баллы
Код запускается без ошибок.	2
Decision Tree обучен с разными max_depth.	2
Random Forest обучен с разными n_estimators.	2
Есть таблица сравнения моделей.	2
Есть письменные выводы по метрикам.	2
Итого	10